Presenter Notes — Week 1–2 Presentation (all 8 slides)

Ground rule for the whole deck: **we present weeks 1–2 only.** What exists so far, officially: repo + toolchain, the TrackerState contract, equipment received and bench-verified, radios flashed with positions visible in the phone app, and the architecture *design*. Firmware implementation, range campaigns, and drive tests are *scheduled*, not claimed. If asked "does it work yet?" → "The foundations are verified — every interface has been proven independently with off-the-shelf tools. Implementation starts on schedule."

The 60-second story arc (whoever opens): *Two vehicles, no infrastructure. Each carries a stock Meshtastic LoRa radio with GPS and a custom sunlight-readable display. Radios share encrypted position broadcasts over 915 MHz; each display reads its own radio over Bluetooth and turns the math into an arrow and a distance. We write exactly one piece of custom firmware — everything radio-hard ships pre-tested. Weeks 1–2 we built the foundations that let four people work in parallel without blocking each other.*

Slide 1 — Title, Objective & Roles (opener)

Say: read the objective sentence, then one line per person — Deo owns the protocol firmware (the BLE client that talks to the radio), Josiah the display and navigation math, Victor radio/antennas/range, Cannon integration and test.

Cheat: objective keywords = *off-grid, continuous direction + distance, multi-km, no infrastructure*. Sponsor: Dr. Richard Gale.

Slide 2 — Semester Timeline, Effort & Budget (Cannon)

Say: 16-week plan. Weeks 1–2 (done): foundations. The lanes run in *parallel* — that's deliberate, enabled by the interface contract on the next slide. Milestones: risk gate W5 (BLE pairing go/no-go), bench demo W7, Demo Day W16.

Cheat: 28 h logged (7 × 4 members) · tracker says 22% overall vs ~13% target — ahead of schedule · risk gate = if BLE pairing fights us two weeks, we swap to Meshtastic's identical API over WiFi/TCP, no redesign.

Slide 3 — Shared Foundations ⭐ DEO

This slide is *why the schedule works*. Three beats:

Beat 1 — the contract. Point at the struct. `TrackerState` is the single interface between my half (protocol) and Josiah's half (display/nav): my side fills in our own GPS fix, the partner's fix, and whether the radio link is up; his side turns it into the arrow. Technical points to land: - **One writer, one reader.** Only the protocol task writes it; only the UI task reads it. One mutex protects it — that eliminates the whole class of ordering/race bugs by construction, not by testing. - **Changed only by joint commit** — it's a governed interface, like an API version, so neither side can break the other silently. - `age_s` on the partner fix is deliberate: in a broadcast system with no acknowledgments, **data age is the honesty metric** the driver sees.

Beat 2 — every interface testable in isolation (the table). Each row is a test that needs *zero of our custom code*: GPS→radio proven on the radio's own OLED and phone app; LoRa A↔B proven with two phones; the BLE Client API

proven from a laptop with Meshtastic's Python CLI; the UI provable against simulated TrackerState scenarios. Punchline: **no one's test depends on someone else's unfinished code — that's what lets four lanes run in parallel.**

Beat 3 — foundations in place (the badges): shared git repo, PlatformIO toolchain for ESP32-S3 (both boards use the same chip → one toolchain for the whole team), nanopb + the Meshtastic protobuf schema compiled, and all equipment received and bench-verified.

Cheat: contract = 1 writer / 1 reader / 1 mutex · tests need 0 custom code · both boards ESP32-S3 · nanopb = tiny C protobuf decoder for microcontrollers. Q: "Why protobuf?" → It's Meshtastic's native wire format — same versioned stream their phone apps use, so it can't silently rot. Q: "Why a struct, not a queue?" → Latest-state semantics: a newer fix supersedes an older one; nobody wants a backlog of stale positions.

Slide 4 — System Architecture (full-page diagram) ⭐ DEO

Walk it **left → right → bottom**, one region at a time:

Left — one vehicle's hardware. GNSS module feeds NMEA over UART into the Heltec V4 (ESP32-S3 + SX1262 LoRa transceiver), which drives a 915 MHz roof antenna over SPI and a small debug OLED over I2C. Below it, the Waveshare 4.2" *reflective* LCD — our display, running the only custom firmware in the system. Both are powered from vehicle 12 V → USB-C, together — powered whenever the vehicle runs. The two links leaving this box: **LoRa 915 MHz** vehicle-to-vehicle, and **BLE GATT** the ~1 m in-cab hop carrying Meshtastic's protobuf stream to the display.

Right — inside the display firmware. Top chain (mine): `ble_link` task — a NimBLE central that connects to the radio, subscribes, and drains the FromRadio queue → `proto_decode` (nanopb) turns bytes into Position/NodeInfo → `node_table` keeps own + partner fixes with timestamps → **writes TrackerState** (the cylinder — same contract from slide 3). Bottom chain (Josiah's): nav engine **reads** TrackerState — haversine distance, initial bearing, heading from GPS course-over-ground (no compass: magnetometers are unreliable inside a steel vehicle) → ui_render draws arrow, distance, age, link status. Sim feed injects fake TrackerState so the UI can be developed with zero radios.

Bottom — the same system as telecom layers. Application: ~30–40 byte position payload, ~13 bit/s useful goodput. Network/MAC: managed flooding, hop limit 3, AES-256, CSMA with SNR-weighted backoff, ~3% channel utilization. PHY: LoRa chirp spread spectrum at 915 MHz, SF11/BW250, FEC 4/5, −129 dBm sensitivity — **decoding 15 dB below the noise floor**. That last number is the whole trick: chirp spread spectrum trades data rate for processing gain, which is why kilometers work on milliwatts.

Close with: *"Vehicle B is an identical copy — the symmetry is the design: no server, no master, no single point of failure."*

Cheat (numbers on this slide): 915 MHz ISM · SF11/BW250 · FEC 4/5 · −129 dBm (15 dB below noise) · ~0.5 s airtime per broadcast · every 30–60 s · hop limit 3 · AES-256 · ~40 B payload · BLE hop ~1 m · display 300×400 1-bit reflective. Q: "Why is the update every 30–60 s and not continuous?" → Each broadcast costs ~0.5 s of airtime at this spreading factor; faster would saturate the shared channel — the design shows data *age* instead of pretending to be real-time. Q: "Why not one custom firmware for everything?" → Everything radio-hard (encryption, mesh relaying, GPS parsing, retries) ships pre-tested inside stock Meshtastic; we write a *client* of its documented API. Smallest possible custom surface = smallest possible risk.

Slide 5 — System Context (Victor)

Say: two identical installs; position broadcasts flow symmetrically — nothing is a server. Any third Meshtastic node (a relay on high ground, a hiker's handheld) extends range with zero configuration because the mesh uses managed flooding. Each display only ever talks to *its own* radio.

Cheat: 30–60 s broadcasts · AES-256 shared channel · hop limit 3 · losing any node degrades only that vehicle's view, never the mesh.

Slide 6 — Hardware Architecture (Victor)

Say: three panels — radio node (GNSS→UART→ESP32-S3→SPI→SX1262→antenna, OLED on I2C for free field debug), display node (ESP32-S3, 16 MB flash / 8 MB PSRAM, SPI framebuffer to the 300×400 reflective LCD), and in-cab power (12 V→USB-C to both, BLE between them). Key line: **the only antenna that leaves the cab is the LoRa antenna** — roof mount is the single biggest range factor. Reflective LCD because a dash in sunlight washes out a TFT, and e-paper is too slow for a moving arrow.

Cheat: one toolchain (both ESP32-S3) · GPS on dash w/ sky view · radio OLED shows sats/nodes/RSSI = built-in debugger.

Slide 7 — End-to-End Data Flow (Josiah)

Say: follow one packet: partner GPS fix (1 Hz NMEA over UART) → Meshtastic B builds a Position packet → LoRa broadcast every 30–60 s (~0.5 s airtime, AES) → Meshtastic A decrypts and queues it → protocol/ drains it over BLE and decodes → TrackerState → nav engine (bearing − own course = arrow angle, haversine = distance) → display redraw. Own position rides the same pipeline minus the LoRa hop. **No ACKs anywhere** — a newer position supersedes an older one, so a lost packet costs one cycle of freshness, never correctness; the driver sees data age instead.

Cheat: end-to-end latency ≈ airtime ≈ 0.5 s — negligible vs the broadcast cadence. Q: "What if a packet is lost?" → nothing breaks; age counts up and the next broadcast heals it.

Slide 8 — Semester Budget (Josiah)

Say: budget summary — the dominant line is engineering labor; direct materials are the four development computers; the radio/display hardware itself is sponsor-provided. Hours: 7 per member logged for weeks 1–2 (28 total), on pace.

Cheat: computers = Direct Material Costs ($3.5k + 3 × $2k = $9.5k) · equipment provided by Dr. Gale · 28 h logged weeks 1–2.

If someone asks a question outside weeks 1–2

"That's scheduled for week N — I can speak to the design intent now and the results when we get there." Then give the design intent from slides 3–4.